

IIF en SQL Server permite realizar evaluaciones condicionales de forma compacta, similar a una función IF en Excel. Es ideal para simplificar CASE simples.

Aquí explico su uso y vemos ejemplos prácticos con la base de datos Northwind:

□ ¿Qué es IIF en SQL Server?

La función IIF evalúa una expresión booleana y devuelve un valor si es verdadera, y otro si es falsa. Su sintaxis es:

```
IIF(condición, valor_si_verdadero, valor_si_falso)
```

Es azúcar sintáctica para CASE, y está disponible desde SQL Server 2012.

📦 Ejemplos con la base de datos Northwind

1. Clasificar productos como 'Caro' o 'Económico'

```
SELECT
    ProductName,
    UnitPrice,
    IIF(UnitPrice > 50, 'Caro', 'Económico') AS CategoriaPrecio
FROM Products;
```

Esto etiqueta los productos según su precio unitario.

2. Ver si un pedido fue enviado o no

```
SELECT
    OrderID,
    CustomerID,
    IIF(ShippedDate IS NOT NULL, 'Enviado', 'Pendiente') AS EstadoEnvio
FROM Orders;
```

Ideal para informes de seguimiento de pedidos.

3. Clientes con o sin fax

```
SELECT
    CompanyName,
    IIF(Fax IS NULL, 'Sin Fax', 'Tiene Fax') AS EstadoFax
FROM Customers;
```

Útil para segmentar clientes según medios de contacto.

4. Descuento aplicado o no en los detalles del pedido

```
SELECT
    OrderID,
    ProductID,
    Discount,
    IIF(Discount > 0, 'Con Descuento', 'Sin Descuento') AS
EstadoDescuento
FROM [Order Details];
```

¿Cuándo usar IIF?

- Para condiciones simples (una sola evaluación booleana).
 - Cuando quieres que tu código sea más legible que con CASE.
 - En columnas calculadas o vistas rápidas.
-

 [Guía práctica: IIF en SQL Server con Northwind](#)

Objetivo

Aprender a usar la función IIF para realizar evaluaciones condicionales simples en consultas SQL, aplicadas a la base de datos Northwind.

Sintaxis general

```
IIF(condición, valor_si_verdadero, valor_si_falso)
```

Ejercicios prácticos

1. Clasificar productos según precio

```
SELECT
    ProductName,
    UnitPrice,
    IIF(UnitPrice >= 50, 'Caro', 'Económico') AS CategoriaPrecio
FROM Products;
```

2. Estado de envío de pedidos

```
SELECT
    OrderID,
    CustomerID,
```

```
IIF(ShippedDate IS NULL, 'Pendiente', 'Enviado') AS EstadoEnvio
FROM Orders;
```

3. Clientes con o sin fax

```
SELECT
    CompanyName,
    IIF(Fax IS NULL, 'Sin Fax', 'Con Fax') AS EstadoFax
FROM Customers;
```

4. Descuento aplicado en detalles de pedido

```
SELECT
    OrderID,
    ProductID,
    Discount,
    IIF(Discount > 0, 'Con Descuento', 'Sin Descuento') AS
EstadoDescuento
FROM [Order Details];
```

5. Empleado activo o no (según fecha de baja ficticia)

```
SELECT
    FirstName,
    LastName,
    IIF(TerminationDate IS NULL, 'Activo', 'Inactivo') AS EstadoEmpleado
FROM Employees;
```

Este ejemplo requiere que agregues una columna ficticia `TerminationDate` para fines didácticos.

Comentario pedagógico

- `IIF` es ideal para ejercicios introductorios de lógica condicional.
- Permite mostrar cómo SQL puede comportarse como Excel en términos de lógica.
- Es más legible que `CASE` en condiciones simples, pero no reemplaza su versatilidad.

Comparativa: `IIF` vs `CASE` en SQL Server

Aspecto	<code>IIF</code>	<code>CASE</code>
Sintaxis	<code>IIF(condición, valor_si_verdadero, valor_si_falso)</code>	<code>CASE WHEN condición THEN valor1 ELSE valor2 END</code>
Complejidad	Simple, solo una condición	Soporta múltiples condiciones y lógica compleja
Legibilidad	Más conciso para condiciones simples	Más claro en lógica ramificada o múltiple

Aspecto	IIF	CASE
Disponibilidad	Desde SQL Server 2012	Disponible en todas las versiones de SQL Server
Tipos de retorno	Puede causar errores si los tipos no coinciden	Más robusto en manejo de tipos
Anidamiento	Limitado, puede volverse ilegible	Más flexible y estructurado

□ Ejemplos paralelos con Northwind

1. Clasificar productos por precio

Con IIF:

```
SELECT
    ProductName,
    UnitPrice,
    IIF(UnitPrice > 50, 'Caro', 'Económico') AS CategoriaPrecio
FROM Products;
```

Con CASE:

```
SELECT
    ProductName,
    UnitPrice,
    CASE
        WHEN UnitPrice > 50 THEN 'Caro'
        ELSE 'Económico'
    END AS CategoriaPrecio
FROM Products;
```

2. Estado de envío de pedidos

Con IIF:

```
SELECT
    OrderID,
    IIF(ShippedDate IS NULL, 'Pendiente', 'Enviado') AS EstadoEnvio
FROM Orders;
```

Con CASE:

```
SELECT
    OrderID,
    CASE
        WHEN ShippedDate IS NULL THEN 'Pendiente'
        ELSE 'Enviado'
    END AS EstadoEnvio
FROM Orders;
```

3. Clasificación múltiple con CASE (no posible con IIF)

```
SELECT
    ProductName,
    UnitPrice,
    CASE
        WHEN UnitPrice >= 100 THEN 'Muy caro'
        WHEN UnitPrice >= 50 THEN 'Caro'
        WHEN UnitPrice >= 20 THEN 'Moderado'
        ELSE 'Económico'
    END AS CategoriaPrecio
FROM Products;
```

Este tipo de lógica no se puede expresar con IIF sin anidamientos complejos y poco legibles.

Comentario pedagógico

- Usá IIF para introducir lógica condicional de forma rápida y visual.
 - Mostrá CASE como herramienta más robusta para lógica compleja.
 - Comparar ambos ayuda a entender cuándo conviene cada uno.
 - Ideal para ejercicios de refactorización: convertir IIF en CASE y viceversa.
-

Ejercicios de conversión: IIF ↔ CASE

Objetivo

Practicar la transformación entre expresiones IIF y estructuras CASE, entendiendo cuándo usar cada una y cómo mantener la lógica intacta.

Ejercicio 1: Clasificación por precio

Versión con IIF:

```
SELECT
    ProductName,
    UnitPrice,
    IIF(UnitPrice > 50, 'Caro', 'Económico') AS CategoriaPrecio
FROM Products;
```

Convertir a CASE:

```
SELECT
    ProductName,
```

```

UnitPrice,
CASE
    WHEN UnitPrice > 50 THEN 'Caro'
    ELSE 'Económico'
END AS CategoriaPrecio
FROM Products;

```

❑  *Comentario:* Ambas versiones son equivalentes. CASE es más flexible si luego se agregan más rangos.

❑ **Ejercicio 2: Estado de envío**

Versión con IIF:

```

SELECT
    OrderID,
    IIF(ShippedDate IS NULL, 'Pendiente', 'Enviado') AS EstadoEnvio
FROM Orders;

```

Convertir a CASE:

```

SELECT
    OrderID,
    CASE
        WHEN ShippedDate IS NULL THEN 'Pendiente'
        ELSE 'Enviado'
    END AS EstadoEnvio
FROM Orders;

```

❑  *Comentario:* Ideal para mostrar cómo se evalúan condiciones booleanas simples.

❑ **Ejercicio 3: Descuento aplicado**

Versión con CASE:

```

SELECT
    OrderID,
    Discount,
    CASE
        WHEN Discount > 0 THEN 'Con Descuento'
        ELSE 'Sin Descuento'
    END AS EstadoDescuento
FROM [Order Details];

```

Convertir a IIF:

```

SELECT
    OrderID,
    Discount,
    IIF(Discount > 0, 'Con Descuento', 'Sin Descuento') AS
EstadoDescuento

```

```
FROM [Order Details];
```

❏  *Comentario:* Buen ejemplo para mostrar cómo `IIF` puede simplificar condiciones directas.

❏ **Ejercicio 4: Clasificación múltiple (solo posible con `CASE`)**

```
SELECT
    ProductName,
    UnitPrice,
    CASE
        WHEN UnitPrice >= 100 THEN 'Muy caro'
        WHEN UnitPrice >= 50 THEN 'Caro'
        WHEN UnitPrice >= 20 THEN 'Moderado'
        ELSE 'Económico'
    END AS CategoriaPrecio
FROM Products;
```

❏  *Comentario:* Este ejercicio muestra por qué `CASE` es necesario cuando hay más de dos condiciones.

Oracle no tiene la función `IIF` como SQL Server, pero su equivalente funcional es `CASE`. También podés usar expresiones `DECODE` para condiciones simples.

Aquí explico cómo replicar `IIF` en Oracle con ejemplos prácticos usando la base de datos HR:

[Equivalentes a `IIF` en Oracle](#)

1. `CASE` (recomendado)

```
CASE WHEN condición THEN valor_si_verdadero ELSE valor_si_falso END
```

2. `DECODE` (solo para igualdad)

```
DECODE(condición, valor_comparado, valor_si_verdadero, valor_si_falso)
```

[Ejemplos con la base HR de Oracle](#)

◆ Clasificar empleados por salario

Con `CASE`:

```

SELECT
  first_name,
  salary,
  CASE
    WHEN salary > 10000 THEN 'Alto'
    ELSE 'Normal'
  END AS categoria_salarial
FROM employees;

```

Con **DECODE** (solo si hay igualdad):

```

SELECT
  first_name,
  salary,
  DECODE(salary, 10000, 'Exacto', 'Otro') AS estado_salarial
FROM employees;

```

◆ Estado de comisión

```

SELECT
  first_name,
  commission_pct,
  CASE
    WHEN commission_pct IS NULL THEN 'Sin comisión'
    ELSE 'Con comisión'
  END AS estado_comision
FROM employees;

```

◆ Clasificación por departamento

```

SELECT
  first_name,
  department_id,
  CASE
    WHEN department_id = 90 THEN 'Executive'
    WHEN department_id = 60 THEN 'IT'
    ELSE 'Otro'
  END AS tipo_departamento
FROM employees;

```

□ Comentario pedagógico

- *CASE es más versátil que DECODE*, ya que permite múltiples condiciones y operadores lógicos.
 - Se pueden usar estos ejemplos para ejercicios de conversión entre **IIF** (SQL Server) y **CASE** (Oracle), mostrando cómo adaptar la lógica entre motores.
-